

Vivado AI Assistant — Tool Reference



MCP server capabilities, tool reference, and documentation chat — Early
Access

August 03, 2026

Copyright © 2026 Advanced Micro Devices, Inc.

Contents

[Vivado MCP Server and Agent Skills](#)

[MCP Tool Reference](#)

Vivado MCP Server and Agent Skills

The Vivado MCP Server connects to AMD documentation, drives a live Vivado session, and automates entire FPGA workflows — all through natural language.

How It Works — Closed-Loop Automation

The agent executes commands, hits an error, searches docs for the fix, applies it, and verifies — automatically.

FPGA Engineer AI Agent Vivado (Live) Knowledge Base "Run synthesis on my design" 1 vivado_execute launch_runs synth_1 2 ERROR: [Synth 8-3352] 3 CLOSED LOOP vivado_doc_search "Synth 8-3352 resolution" 4 KBA with fix and Tcl command 5 vivado_execute apply fix 6 synthesis complete · 0 errors 7 "Fixed the error — synthesis passed" 8 FPGA Engineer AI Agent Vivado (Live) Knowledge Base

Capabilities

- **Documentation Search** — RAG-powered search across 15+ User Guides, Application Notes, and Knowledge Base Articles
- **Session Control** — Start, stop, and monitor Vivado sessions (TCL or GUI mode)
- **Design Execution** — Run any Tcl command through a standardized MCP API
- **Agent Skills** — Expert-authored workflow files that encode FPGA methodology. Use the bundled example designs to get started; official standalone skills are coming in upcoming releases.
- **Documentation Chat** — The same knowledge base is also available as a standalone web chat at vivado.amd.com/chat — no setup required.

What's Included

agentic-ai-suite.zip

Agent skills, getting started guides, and the MCP tools reference.

Category	Skill / Content	Description
Timing	timing-methodology-checks	Run 55+ timing methodology checks (UG906) on synthesized designs, identify violations, and generate actionable RTL/XDC fixes
Revision Control	vivado-revision-control	Automate revision control setup — detect project type, analyze sources, export components, capture settings, and generate a build script for reproducible project recreation
Hardware Debug	hw-ila-debug	Interact with ILA debug cores on live hardware — discover cores, configure triggers, arm, capture, upload, and export waveform data (CSV/VCD)
Hardware Debug	hw-noc-debug	Debug Versal NoC issues — autonomous pipeline: connect, scan, design correlation, root-cause analysis for AXI failures, decode errors, and timeouts
Hardware Debug	hw-vio-debug	Interact with VIO debug cores on live hardware — read input probes, drive output probes, monitor activity, and reset outputs
Setup	Getting Started Guides	Setup guides for VS Code, Cursor, and CLI tools
Reference	MCP Tools Reference	Complete reference for all Vivado MCP tools

vivado-ai-assistant-examples-0.6.8.zip

Self-contained example workspaces with design files, agent skills, and step-by-step walkthroughs.

Category	Example	Description
Design Capture	Design Creation Prototype	AI-guided project creation, IP instantiation, and block design assembly
Design Analysis	RTL Lint	Run <code>synth_design -lint</code> to catch RTL issues before synthesis
Design Analysis	Multi-Run Analysis	Compare results across multiple implementation runs

Category	Example	Description
Design Analysis	Opt Design Log Analysis	Parse and analyze optimization logs for actionable insights
Design Closure	Timing Closure Prototype	End-to-end timing analysis and constraint fixing
Design Closure	AXI4 Debug Simulation	Debug AXI4 protocol issues through AI-guided simulation
Hardware Debug	ILA & VIO Insertion Flow	Insert AXIS-ILA and AXIS-VIO debug cores into a Versal VCK190 block design

MCP Tool Reference

Complete reference for all tools provided by the Vivado MCP Server. These are the capabilities available to any MCP-compatible AI agent (VS Code Copilot, Cursor, Claude Code, etc.) when connected to the server.

Quick Reference

Tool	Category	Description
<code>vivado_start</code>	Session	Launch a new Vivado session
<code>vivado_stop</code>	Session	Stop a Vivado session
<code>vivado_connect</code>	Session	Reconnect to an existing Vivado instance
<code>vivado_list_sessions</code>	Session	List all active sessions
<code>vivado_cleanup</code>	Session	Clean up stale sessions and orphaned processes
<code>vivado_execute</code>	Execution	Execute Tcl commands in Vivado
<code>vivado_status</code>	Monitoring	Check session health and run progress
<code>vivado_log_messages</code>	Monitoring	Parse log for errors, warnings, and messages
<code>vivado_history</code>	Monitoring	Get or search command history
<code>vivado_display</code>	GUI	Manage displays, screenshots, and virtual displays (Linux only)
<code>vivado_doc_search</code>	Research	Search AMD/Xilinx documentation
<code>vivado_ssh</code>	Remote	Run Vivado on remote machines via SSH (Linux only)
<code>vivado_todos</code>	Workflow	Track multi-step task progress in IDE sidebar
<code>vivado_feedback</code>	Workflow	Collect user feedback after tasks
<code>vivado_client_info</code>	Workflow	Get MCP client capabilities

Session Management

`vivado_start`

Launch a new Vivado session. Always starts in Tcl mode by default — GUI mode is only used when the user explicitly requests graphical interaction. The server configures a Tcl webserver and smart proxy for command execution.

Parameter	Type	Required	Description
<code>working_dir</code>	string	Yes	Working directory for the Vivado project. Must exist and be writable
<code>session_type</code>	enum	Yes	<code>ipi</code> for block design workflows, <code>general</code> for everything else
<code>gui_mode</code>	boolean	No	<code>true</code> for GUI mode with bundled VNC display. Default: <code>false</code> (Tcl mode)
<code>display_mode</code>	enum	No	Display handling: <code>auto</code> (default), <code>vnc</code> , <code>x11</code> , <code>none</code>
<code>display</code>	string	No	X11 display string (e.g. <code>:99</code>). Required when <code>display_mode=x11</code>
<code>port</code>	number	No	Webserver port. Default: auto-assigned
<code>vivado_path</code>	string	No	Path to Vivado executable. Auto-resolved from PATH if omitted
<code>env</code>	object	No	Extra environment variables for the Vivado process

Tips: - Sessions start in Tcl mode (headless) by default. Request GUI mode only when you need visual interaction (block designs, schematics). - You can start in Tcl mode and switch to GUI later with `start_gui` — no restart needed. - In GUI mode, the server spins up a VNC display automatically with a VNC connection string. - Use `ipi` session type for Block Design (IP Integrator) workflows, `general` for everything else.

`vivado_stop`

Gracefully stop a Vivado session. Saves state, closes the project, and terminates the Vivado process.

Parameter	Type	Required	Description
<code>session_id</code>	string	Yes	Session to stop
<code>close_vivado</code>	boolean	No	Close Vivado application. Default: <code>true</code>

Set `close_vivado=false` to detach the MCP server while keeping Vivado running.

vivado_connect

Reconnect to an existing Vivado instance that already has a running Tcl webserver.

Parameter	Type	Required	Description
<code>webserver_url</code>	string	Yes	URL of the Vivado webserver (e.g. <code>http://localhost:8088</code>)
<code>proxy_addr</code>	string	No	Smart proxy address (<code>host:port</code>). Preferred for better performance
<code>session_id</code>	string	No	Reconnect to a known session by ID
<code>pid</code>	integer	No	Verify process is alive before connecting
<code>working_dir</code>	string	No	Working directory reference

vivado_list_sessions

List all active Vivado sessions with detailed metadata. No parameters required.

Returns for each session: `session_id`, `session_type`, `deployment_type` (`local`, `ssh`), `vivado_version`, `vivado_edition`, `is_gui_mode`, `hostname`, `uptime_seconds`, `proxy_addr`, and SSH details (`ssh_host`, `ssh_user`).

vivado_cleanup

Clean up stale sessions and orphaned processes.

Parameter	Type	Required	Description
<code>operation</code>	enum	Yes	See operations below
<code>session_id</code>	string	No	Target specific session

Operations:

Operation	Description
<code>stats</code>	Session count and statistics

Operation	Description
<code>health</code>	Check all session health
<code>cleanup_stale</code>	Remove dead sessions from tracking
<code>kill_orphans</code>	Kill Vivado processes started by MCP but not tracked
<code>kill_all</code>	Kill all MCP-started Vivado processes
<code>recover</code>	Reconnect to persisted sessions

Safe by design — only affects Vivado processes started by the MCP server. It will never touch Vivado instances you launched manually.

Command Execution

`vivado_execute`

Execute Tcl commands in Vivado. This is the primary tool for all Vivado operations.

Parameter	Type	Required	Description
<code>command</code>	string	Yes	Tcl command(s) to execute. Batch with semicolons
<code>session_id</code>	string	Yes	Target session
<code>mode</code>	enum	No	<code>auto</code> (default), <code>proxy</code> , or <code>webserver</code>
<code>capture_log</code>	boolean	No	Capture log output. Default: <code>true</code>
<code>echo</code>	boolean	No	Show command in Vivado console. Default: <code>true</code>

Batching commands: Multiple setup commands can be combined with semicolons:

```
add_files src/top.v; set_property top top [current_fileset];
update_compile_order -fileset sources_1
```

Standalone commands (must appear first or alone): - Design flow: `synth_design`, `place_design`, `route_design`, `opt_design`, `phys_opt_design`, `write_bitstream` - Run management: `launch_runs`, `wait_on_run`, `reset_runs` - Project: `open_project`, `close_project`, `open_checkpoint`, `read_checkpoint`, `source` - GUI: `start_gui`, `stop_gui`

Synthesis & Implementation flow:

```
-- Step 1: launch_runs synth_1 -jobs 8
-- Step 2: wait_on_run synth_1
-- Step 3: launch_runs impl_1 -to_step write_device_image -jobs 8
-- Step 4: wait_on_run impl_1
```

Long-running commands: Commands like synthesis or implementation can take minutes to hours. The server hands off execution and monitors progress in the background. The agent automatically polls `vivado_status` to track progress. You can check `vivado_log_messages` at any time to see errors or warnings as they appear.

Monitoring

vivado_status

Check session health, monitor synthesis/implementation runs, and recover stuck processes.

Parameter	Type	Required	Description
session_id	string	Yes	Target session
action	enum	No	session (default), runs , health , reset_run , cancel_and_relaunch
run_name	string	No	Run to reset (for reset_run)
jobs	integer	No	Parallel jobs (for cancel_and_relaunch)

Can be called while a command is still running — it doesn't interfere with execution.

vivado_log_messages

Parse the Vivado log file for structured error, warning, critical warning, and info messages.

Parameter	Type	Required	Description
session_id	string	Yes	Target session
filter	string	No	errors , warnings , critical , info , all . Default: errors + critical + warnings

Parameter	Type	Required	Description
<code>max_messages</code>	number	No	Limit per category. Default: all

Can be called at any time, even during long-running commands.

`vivado_history`

Get or search the complete command history for a session.

Parameter	Type	Required	Description
<code>session_id</code>	string	Yes	Target session, or <code>all</code> to search across all sessions
<code>search_query</code>	string	No	Filter by command text or output (case-insensitive)
<code>limit</code>	integer	No	Max entries. Default: 50
<code>since_id</code>	integer	No	Only entries after this history ID

GUI & Display

`vivado_display`

Linux only. `vivado_display` is not supported on Windows.

Manage X11 displays, capture screenshots, and run virtual displays (TigerVNC Xvnc) for headless GUI operation.

Parameter	Type	Required	Description
<code>action</code>	enum	Yes	<code>check</code> , <code>screenshot</code> , <code>virtual_start</code> , <code>virtual_stop</code> , <code>virtual_status</code>
<code>session_id</code>	string	Conditional	Required for <code>screenshot</code>
<code>display_num</code>	number	No	Display number for <code>virtual_start</code>
<code>width</code>	number	No	Screen width. Default: 1920
<code>height</code>	number	No	Screen height. Default: 1080

Native display vs. virtual display: By default, the agent may open the GUI on a virtual VNC display rather than your native X11 display. To ensure Vivado opens on your display:

1. Check your display: `echo $DISPLAY` in your terminal (e.g. `:0` , `localhost:10.0`)
 2. Tell the agent explicitly: *"Open the GUI on my native display"* or *"Use display :0"*
 3. Or pass `display_mode='x11'` when starting Vivado via `vivado_start` or `vivado_ssh`
-

Research

`vivado_doc_search`

Search the AMD/Xilinx documentation database using hybrid search (BM25 keyword matching + vector semantic search).

Parameter	Type	Required	Description
<code>query</code>	string	Yes	Search query
<code>top_k</code>	number	No	Number of results. Default: 25
<code>alpha</code>	number	No	Semantic vs keyword weight (0.0-1.0). Default: 0.80

Use lower alpha values (0.3-0.5) for exact Tcl command lookups. Use higher values (0.7-0.9) for conceptual questions.

Remote Deployment

`vivado_ssh`

Linux only. `vivado_ssh` is not supported on Windows.

Start and manage Vivado sessions on remote machines via SSH. The remote machine must share the same filesystem (NFS) as the local machine.

Parameter	Type	Required	Description
<code>action</code>	enum	Yes	<code>start</code> , <code>status</code> , or <code>kill</code>
<code>ssh_host</code>	string	Conditional	Remote hostname. Required for <code>start</code>
<code>working_dir</code>	string	Conditional	Project directory on shared filesystem. Required for <code>start</code>

Parameter	Type	Required	Description
<code>session_type</code>	enum	Conditional	<code>general</code> or <code>ipi</code> . Required for <code>start</code>
<code>gui_mode</code>	boolean	No	Launch with GUI. Default: <code>false</code>
<code>display_mode</code>	enum	No	Display management: <code>auto</code> , <code>vnc</code> , <code>x11</code> , <code>none</code> . Default: <code>auto</code>
<code>ssh_user</code>	string	No	SSH username. Default: current user
<code>ssh_key</code>	string	No	Path to SSH private key
<code>session_id</code>	string	No	Session ID (for <code>status</code> / <code>kill</code>)

SSH deployment requires the working directory to be on a shared filesystem (NFS) accessible at the same path on both machines.

Workflow

`vivado_todos`

Track multi-step task progress with a structured todo list. Tasks appear in the IDE sidebar.

Parameter	Type	Required	Description
<code>operation</code>	enum	Yes	<code>plan</code> , <code>update</code> , <code>read</code> , <code>clear</code> , <code>show</code>
<code>tasks</code>	array	Conditional	Task list for <code>plan</code>
<code>title</code>	string	Conditional	Plan title. Required for <code>plan</code>
<code>id</code>	string	Conditional	Task ID. Required for <code>update</code>
<code>status</code>	enum	Conditional	<code>active</code> , <code>done</code> , <code>skipped</code> , <code>failed</code> . Required for <code>update</code>

`vivado_feedback`

Collect user feedback after completing a significant task.

Parameter	Type	Required	Description
<code>action</code>	enum	Yes	<code>collect</code> or <code>submit</code>

Parameter	Type	Required	Description
<code>task_summary</code>	string	Conditional	What was accomplished. Required for <code>collect</code>
<code>rating</code>	enum	Conditional	<code>good</code> , <code>neutral</code> , <code>poor</code> . Required for <code>submit</code>
<code>feedback_text</code>	string	No	Free-text comments

`vivado_client_info`

Get information about the connected MCP client and its capabilities. No parameters. Returns client name, version, and supported features (GUI, terminal, sidebar).

Parallel Execution

Read-only tools (`vivado_status` , `vivado_log_messages` , `vivado_history` , `vivado_doc_search` , `vivado_list_sessions`) can all run in parallel — even while a command is executing.

Command execution (`vivado_execute`) is sequential within a single session because Vivado's Tcl interpreter is single-threaded. However, commands on **different sessions** can run in parallel.

Disabling Tools

If a tool causes unwanted behavior, you can disable individual MCP tools in your IDE.

VS Code / GitHub Copilot: In the Chat view, select the **Configure Tools** button (wrench icon) in the chat input field. Toggle specific tools on or off. To disable an entire MCP server, right-click it in the **MCP SERVERS - INSTALLED** section and select **Disable**.

Cursor: Use the tools picker in the chat input. Toggle individual tools on or off before sending your prompt.

GitHub Copilot CLI:

```
copilot --deny-tool='vivado(vivado_display)'
```

Claude Code: Add a deny rule to your `~/.claude/settings.json` or `.claude/settings.json` :

```
{
  "permissions": {
    "deny": ["mcp__vivado__vivado_display"]
  }
}
```

OpenAI Codex CLI: Add `disabled_tools` to the server entry in `~/.codex/config.toml`:

```
[mcp_servers.vivado-mcp]
command = "/path/to/vivado-mcp-server"
args = ["--stdio-bridge"]
disabled_tools = ["vivado_display"]
```